UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA

PROYECTO FINAL

DESARROLLO WEB

VIVENT

Sistema SaaS multiempresa de gestión de inventarios

Documentación del proyecto: factibilidades, requerimientos, diagramas UML, modelo entidad/relación, diseño arquitectónico, diseño de componentes, diseño de interfaz y experiencia de usuario, evaluación del diseño y refactorización aplicada.

I N T E G R A N T E S

Christian Méndez

Wilfero Delgado

Fernando Palencia

Guatemala, octubre de 2026

Versión 1 · Incluye los tres procesos integrados, las tres APIs externas y la colección Postman con CRUD completo, verificada con 128 requests y 242 aserciones sin fallos.

Índice

#	Sección
1	Negocio y giro
2	Problema
3	Objetivos
4	Alcance
5	Los seis módulos del negocio
6	Reglas del negocio (30)
7	Diseño del sistema
7.1	Modelo conceptual
7.2	Modelo de datos físico
7.3	Arquitectura
7.4	Flujos de los tres procesos integrados
8	Matriz de responsabilidades
9	Stack tecnológico y equivalencia
10	Repositorio, Docker y publicación
11	API, colección Postman y CRUD
11.1	Instalación de la colección
11.2	Variables de la colección
11.3	Estructura de la colección
11.4	Matriz CRUD por recurso
11.5	Corrida encadenada de una prueba completa
11.6	Errores controlados y resultado de la corrida
12	Anexos
13	Rúbrica

Curso: Desarrollo Web · **Catedrático:** M.A. Ing. Elvis Morales · **Ciclo:** octubre 2026

Repositorio académico: `https://github.com/wilfero/vivent-academico` (privado; el docente recibe acceso) · **Entrega:** 25 de octubre de 2026

1. Negocio y giro

Vivent atiende **empresas comerciales y distribuidoras pequeñas** de Guatemala: tiendas de barrio, distribuidoras de bebidas y abarrotes, papelerías y comedores que manejan mercadería física y, en varios casos, más de una sucursal.

El negocio modelo de la demostración es **Demo Vivent GT** (`demo.corpoviva.com`): una distribuidora de bebidas, snacks, abarrotes, productos de limpieza y papelería, con dos sucursales (Main Branch y Demo Branch 2), un almacén principal por sucursal, seis usuarios con distintos roles y operaciones diarias de compra, venta, traslado y ajuste.

Vivent es **SaaS multiempresa**: cada compañía se identifica por subdominio, tiene su propio color, logo, idioma (es-GT / en-US), moneda (GTQ por defecto) y zona horaria, y sus datos están aislados de los de cualquier otra compañía.

2. Problema

En este tipo de empresas el inventario se administra con cuadernos, hojas de cálculo o sistemas que solo registran la venta y no el costo del producto. Las consecuencias típicas:

- **No se sabe cuánto hay ni dónde está.** El conteo físico descubre diferencias sin explicación ni responsable.
- **El costo de lo vendido se calcula a mano.** Al no controlar capas de costo, la utilidad del mes es una estimación.
- **Las correcciones no dejan evidencia.** Daños, pérdidas y vencimientos se anotan sueltos o no se anotan.
- **El traslado entre sucursales es informal.** Se despacha por teléfono, se recibe de menos y nadie sabe qué faltó.
- **El acceso es total o nulo.** Todos ven y editan todo, sin permisos por rol ni bitácora.
- **La caja no cuadra y no se puede auditar.** No existe un cierre por usuario y sucursal con diferencia calculada.

3. Objetivos

Objetivo general. Desarrollar un sistema web que registre las operaciones de inventario de una empresa comercial multiempresa, aplicando reglas del negocio verificadas en el backend y dejando trazabilidad completa de cada movimiento.

Objetivos específicos.

1. Registrar compras y ventas por cualquier presentación, convirtiendo siempre a unidad base.
2. Costear el inventario con **FIFO/PEPS por capas** y reflejar todo movimiento en el **kardex**.
3. Controlar el acceso con **roles y permisos** `módulo.acción`, aplicados también en la API.
4. Operar **traslados entre sucursales** con reserva, despacho y recepción total o parcial.
5. Corregir el inventario con **ajustes** por motivo y aprobación.
6. Generar **nueve reportes** con filtros y exportación, más un dashboard con indicadores reales.

7. **Auditar** las acciones sensibles y conservar la trazabilidad con eliminación lógica.
8. Publicar base de datos, backend y frontend en la nube y demostrar un proceso completo en línea.

4. Alcance

4.1 Incluye

Área	Alcance entregado
Autenticación	Login con `subdominio + usuario + contraseña` validado en el backend; **access token JWT (30 min) + refresh token (7 días)** con rotación, renovación y revocación en logout.
Usuarios y permisos	6 roles del sistema, permisos `módulo.acción`, overrides GRANT/DENY por usuario; restricciones replicadas en la API.
Dashboard	Indicadores reales calculados desde la base de datos.
Módulos del negocio	6 módulos completos (de 11 disponibles).
Procesos integrados	3 procesos, uno por integrante, cada uno conectando al menos dos módulos.
Consultas	Búsqueda por texto, filtros combinables por módulo y ordenamiento por columnas.
Reportes	9 reportes con filtros y exportación.
Validaciones	Frontend (Zod + React Hook Form) y backend (Bean Validation + reglas de servicio), con mensajes comprensibles.
APIs externas	3 integraciones reales (Photon, Open Food Facts, tipo de cambio).
Publicación	Guía y configuración para BD + backend + frontend en la nube, con cuentas de prueba.

4.2 No incluye (fase 2)

Facturación legal, integración tributaria, pagos automáticos reales, adjuntos S3, aplicación móvil, e-commerce público y cuentas por pagar/cobrar detalladas. La arquitectura no bloquea su incorporación posterior.

5. Los seis módulos del negocio

> Los módulos de login, dashboard y configuración general no se contabilizan, conforme al enunciado.

5.1 Catálogo (responsable: Christian Méndez)

- **Propósito:** definir qué se compra y se vende, con su unidad base, presentaciones y códigos de barras.
- **Actores:** COMPANY_ADMIN, BRANCH_ADMIN, MANAGER (lectura para vendedores).
- **Entidades:** `product_units`, `categories`, `products`, `product_presentations`.
- **Operaciones:** alta, edición y desactivación lógica de unidades, categorías, productos y presentaciones; consulta por código de barras.

- **Pantallas:** Unidades, Categorías, Productos (con su detalle de presentaciones), buscador de catálogo.
- **Reglas aplicadas:** R11, R12, R13, R29.
- **Permisos:** `units.*`, `product_categories.*`, `products.*`, `product_presentations.*`.

5.2 Inventario (responsable: Christian Méndez)

- **Propósito:** conocer la existencia real por sucursal y ubicación, su costo por capas y su historial.
- **Entidades:** `inventory_balances`, `inventory_layers`, `stock_movements`, `locations`.
- **Operaciones:** consulta de existencias por sucursal y global, kardex por producto, ajuste manual controlado del stock, consulta de capas FIFO activas.
- **Reglas aplicadas:** R14 a R18, R25, R28.
- **Permisos:** `inventory.view`, `inventory.view_kardex`, `inventory.adjust`, `inventory.adjust_approve`.

5.3 Compras (responsable: Wilfero Delgado)

- **Propósito:** ingresar mercadería de proveedores y crear las capas de costo.
- **Entidades:** `purchases`, `purchase_details`, `suppliers`.
- **Estados:** `DRAFT` → `CONFIRMED` → `VOIDED` (y `PENDING_APPROVAL` opcional).
- **Operaciones:** crear borrador, editar borrador, confirmar, listar con filtros, ver detalle, anular con motivo.
- **Reglas aplicadas:** R14, R15, R19, R20, R28.
- **Permisos:** `purchases.view`, `purchases.create`, `purchases.update`, `purchases.confirm`, `purchases.void`.

5.4 Ventas POS (responsable: Wilfero Delgado)

- **Propósito:** vender al mostrador con escaneo de código de barras, descuentos, cliente y método de pago.
- **Entidades:** `sales`, `sale_details`, `customers`.
- **Estados:** `DRAFT` → `CONFIRMED` → `CANCELLED`.
- **Operaciones:** abrir venta, identificar cliente (o consumidor final), escanear o buscar presentación, ajustar cantidades, aplicar descuentos, finalizar, anular con motivo y disposición de productos, consultar historial.
- **Reglas aplicadas:** R16, R17, R18, R21, R27, R28.
- **Permisos:** `sales.view`, `sales.create`, `sales.cancel`, `sales.apply_discount`, `sales.remove_item`, `sales.supervisor_authorize`.

5.5 Traslados (responsable: Fernando Palencia)

- **Propósito:** mover mercadería entre sucursales de la misma compañía con control y evidencia.
- **Entidades:** `transfers`, `transfer_details` + reservas en `inventory_balances`.
- **Estados:** `REQUESTED` → `APPROVED` → `IN_TRANSIT` → `RECEIVED / PARTIALLY_RECEIVED`, con `REJECTED` y `CANCELLED`.
- **Operaciones:** solicitar, aprobar (reserva), rechazar, despachar, recibir total o parcial, cancelar con motivo.
- **Separación de funciones:** la aprobación exige `transfers.approve` (MANAGER o COMPANY_ADMIN); el despacho exige `transfers.dispatch` (WAREHOUSE_USER). En la demo,

`transfers_destination` solicita y recibe, `admin` aprueba y `transfers_origin` despacha: así se demuestra que un usuario sin permiso recibe 403.
- **Reglas aplicadas:** R10, R22, R23, R24, R28.
- **Permisos:** `transfers.view`, `transfers.request`, `transfers.approve`, `transfers.dispatch`, `transfers.receive`, `transfers.cancel`.

5.6 Ajustes (responsable: Fernando Palencia)

- **Propósito:** corregir el inventario con motivo, dirección y aprobación.
- **Entidades:** `inventory_adjustments`, `inventory_adjustment_lines`.
- **Tipos:** `PHYSICAL_COUNT`, `DAMAGE`, `LOSS`, `THEFT`, `EXPIRED`, `INTERNAL_USE`, `REGISTER_ERROR`, `SUPPLIER_RETURN`, `CUSTOM`. **Dirección:** `IN` / `OUT`.
- **Estados:** `DRAFT`, `PENDING_APPROVAL`, `APPROVED`, `REJECTED`, `CANCELLED`.
- **Operaciones:** crear, editar borrador, enviar a aprobación, aprobar, rechazar con motivo, cancelar con motivo.
- **Reglas aplicadas:** R25, R28, R29.
- **Permisos:** `inventory.adjust`, `inventory.adjust_approve`.

6. Reglas del negocio (30)

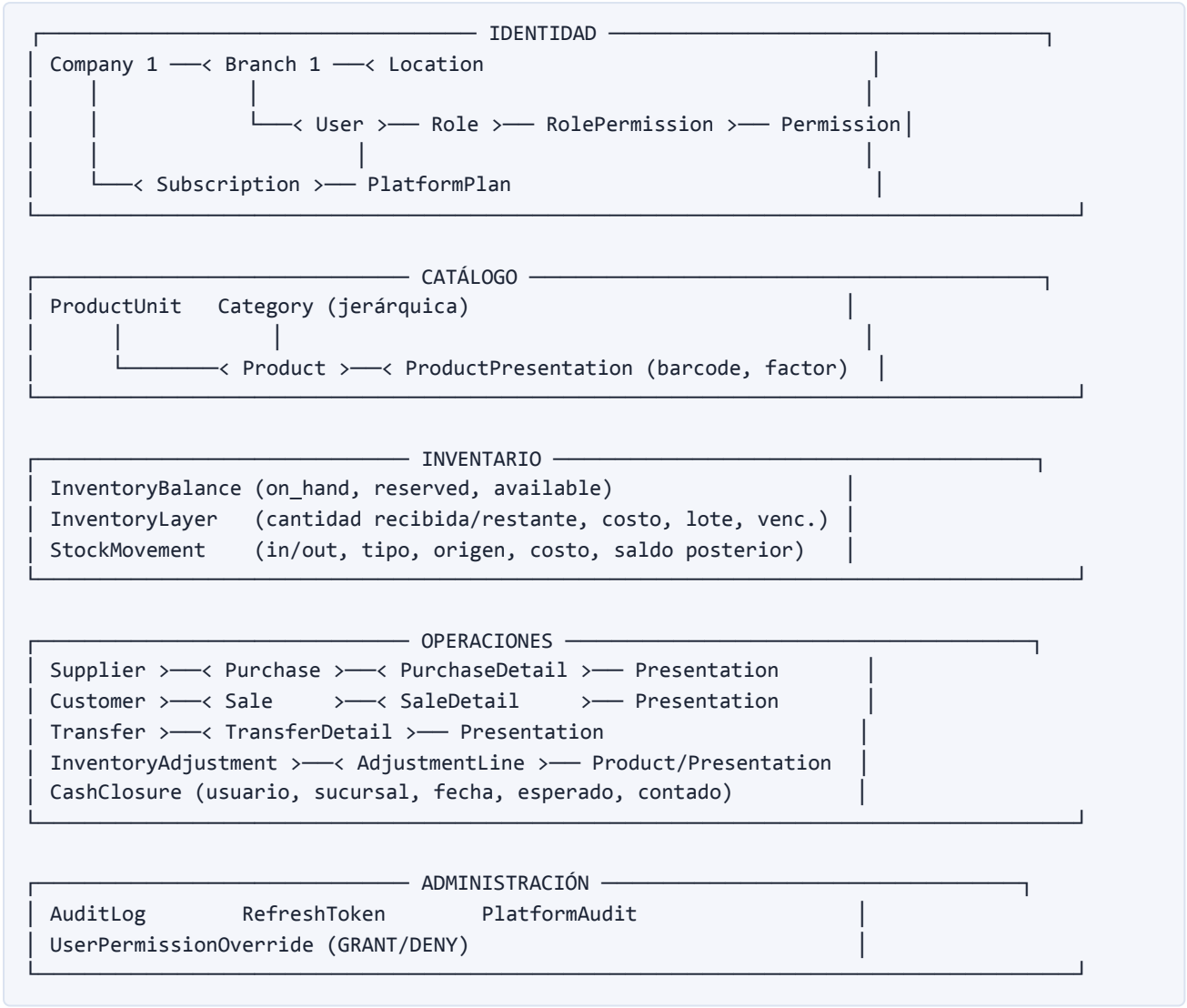
#	Regla	Dónde se hace cumplir
R1	Una compañía no ve ni afecta datos de otra compañía.	Filtro obligatorio por `company_id` en servicios y consultas
R2	Cada compañía se identifica por su subdominio.	Resolución de tenant en login y configuración pública
R3	Un usuario pertenece a una sola compañía.	FK `users.company_id` + validación en token
R4	Un usuario pertenece a una sucursal activa y puede trasladarse a otra de la misma compañía.	`users.branch_id` + historial de asignaciones
R5	El username es único por compañía.	Índice único `(company_id, username)`
R6	El login valida subdominio + usuario + contraseña.	`AuthService.login`
R7	La sesión usa access token (30 min) y refresh token (7 días).	`JwtService` + `RefreshTokenService`
R8	El refresh token rota en cada uso y se revoca al cerrar sesión y al cambiar contraseña.	Tabla `refresh_tokens`, `auth_version`
R9	Los permisos se resuelven por rol y luego por overrides DENY y GRANT.	`PermissionResolutionService`
R10	Un usuario no-COMPANY_ADMIN solo opera su sucursal.	Validación de branch/scope en servicios
R11	Todo producto declara una unidad base.	`products.base_unit_code` obligatorio
R12	Toda presentación se convierte a unidad base con factor > 0.	`product_presentations.conversion_factor` con CHECK
R13	El código de barras pertenece a la presentación y es único por compañía.	Índice único parcial

#	Regla	Dónde se hace cumplir
R14	Compras y ventas se registran por presentación y se convierten a unidad base.	Cálculo en servicios de compra/venta
R15	Al confirmar una compra se crean capas FIFO.	`InventoryLayerService`
R16	La venta consume primero las capas más antiguas con saldo.	`FifoConsumptionService`
R17	`disponible = existencia - reservado` es columna generada.	`inventory_balances` (GENERATED ALWAYS)
R18	No se vende sin stock disponible; el stock nunca es negativo.	CHECK en BD + validación en servicio
R19	La compra confirmada es inmutable; se corrige anulándola con motivo.	Estado `CONFIRMED` sin edición
R20	La anulación de compra revierte exactamente sus capas originales.	Reverso por `purchase_detail_id`
R21	La anulación de venta exige permiso, motivo y disposición de productos.	`SalesService.cancel`
R22	El traslado reserva stock desde la aprobación.	`quantity_reserved` en consolidación
R23	La recepción puede ser parcial y deja evidencia del faltante.	`quantity_approved_base` vs `quantity_received_base`
R24	No se traslada entre compañías ni se recibe más de lo aprobado.	Validación de tenant y cantidades
R25	Los ajustes se aprueban antes de impactar el inventario.	`AdjustmentsService.approve`
R26	El cierre de caja es único por usuario, sucursal y fecha.	Restricción única en BD
R27	Los descuentos requieren permiso y quedan registrados.	`sales.apply_discount` + auditoría
R28	Toda acción sensible se audita con valores antes/después.	`AuditLogService` + tablas de auditoría
R29	Nada se borra físicamente (`deleted_at` / estados).	Repositorios con filtro `deleted_at IS NULL`
R30	No se versionan credenciales; las contraseñas se guardan con BCrypt.	`.gitignore`, `.env.example`, `BCryptPasswordEncoder`

7. Diseño del sistema

7.1 Modelo conceptual

El sistema se organiza en cinco dominios: **identidad**, **catálogo**, **inventario**, **operaciones** y **administración**.



Relaciones principales

Relación	Cardinalidad	Regla asociada
Company → Branch	1 : N	R1, R2
Branch → Location	1 : N (una por defecto)	R10
Company → User	1 : N	R3, R5
Role → Permission	N : M	R9
Product → Presentation	1 : N	R11, R12
Presentation → SaleDetail / PurchaseDetail	1 : N	R13, R14
Purchase → InventoryLayer	1 : N	R15
Sale → StockMovement (SALE_OUT)	1 : N	R16, R18
Transfer → StockMovement (OUT/IN)	1 : N	R22, R23

7.2 Modelo de datos físico

Motor: **PostgreSQL 16/17**. Esquema creado desde cero con migraciones Flyway versionadas (`V*.sql`); datos de prueba con seeds idempotentes (`S*.sql` , historial propio en `flyway_seed_history`).

Tablas núcleo y columnas relevantes

Tabla	Columnas clave	Restricciones
`companies`	`id` , `tenant_slug` , `domain` , `commercial_name` , `currency_code` , `timezone` , `status`	UNIQUE `tenant_slug`
`branches`	`id` , `company_id` , `code` , `name` , `is_active`	UNIQUE `(company_id, code)`
`locations`	`id` , `company_id` , `branch_id` , `code` , `type` , `is_default`	UNIQUE `(branch_id, code)`
`users`	`id` , `company_id` , `branch_id` , `username` , `password_hash` , `role_id` , `auth_version` , `status`	UNIQUE `(company_id, username)`
`permissions`	`id` , `code` (`module.action`), `module_code` , `action_code`	UNIQUE `code`
`role_permissions`	`role_id` , `permission_id`	PK compuesta
`user_permission_overrides`	`user_id` , `permission_id` , `effect`	CHECK `effect IN ('GRANT','DENY')`
`refresh_tokens`	`id` , `user_id` , `company_id` , `token_hash` , `expires_at` , `revoked_at`	UNIQUE `token_hash` , índice por usuario
`product_units`	`code` (PK), `name` , `allows_decimal`	CHECK factor decimal
`categories`	`id` , `company_id` , `parent_category_id` , `name`	UNIQUE `(company_id, name)`
`products`	`id` , `company_id` , `base_unit_code` , `sku` , `minimum_stock` , `is_stockable`	FK unidad base, UNIQUE `(company_id, sku)`
`product_presentations`	`id` , `product_id` , `barcode` , `conversion_factor` , `sale_price`	CHECK `conversion_factor > 0` , UNIQUE barcode
`inventory_balances`	`company_id` , `branch_id` , `location_id` , `product_id` , `quantity_on_hand` , `quantity_reserved` , `quantity_available` (generada)	CHECK `on_hand >= 0` , `reserved >= 0`
`inventory_layers`	`id` , `product_id` , `quantity_received` , `quantity_remaining` , `unit_cost` , `purchase_detail_id` , `expiration_date` , `lot_number`	CHECK `remaining >= 0`
`stock_movements`	`id` , `movement_type` , `source_type` , `source_id` , `quantity_in` , `quantity_out` , `unit_cost` , `balance_after`	Índice por producto/fecha
`purchases`	`id` , `branch_id` , `supplier_id` , `purchase_number` , `status` , `total_amount`	UNIQUE `(company_id, purchase_number)`

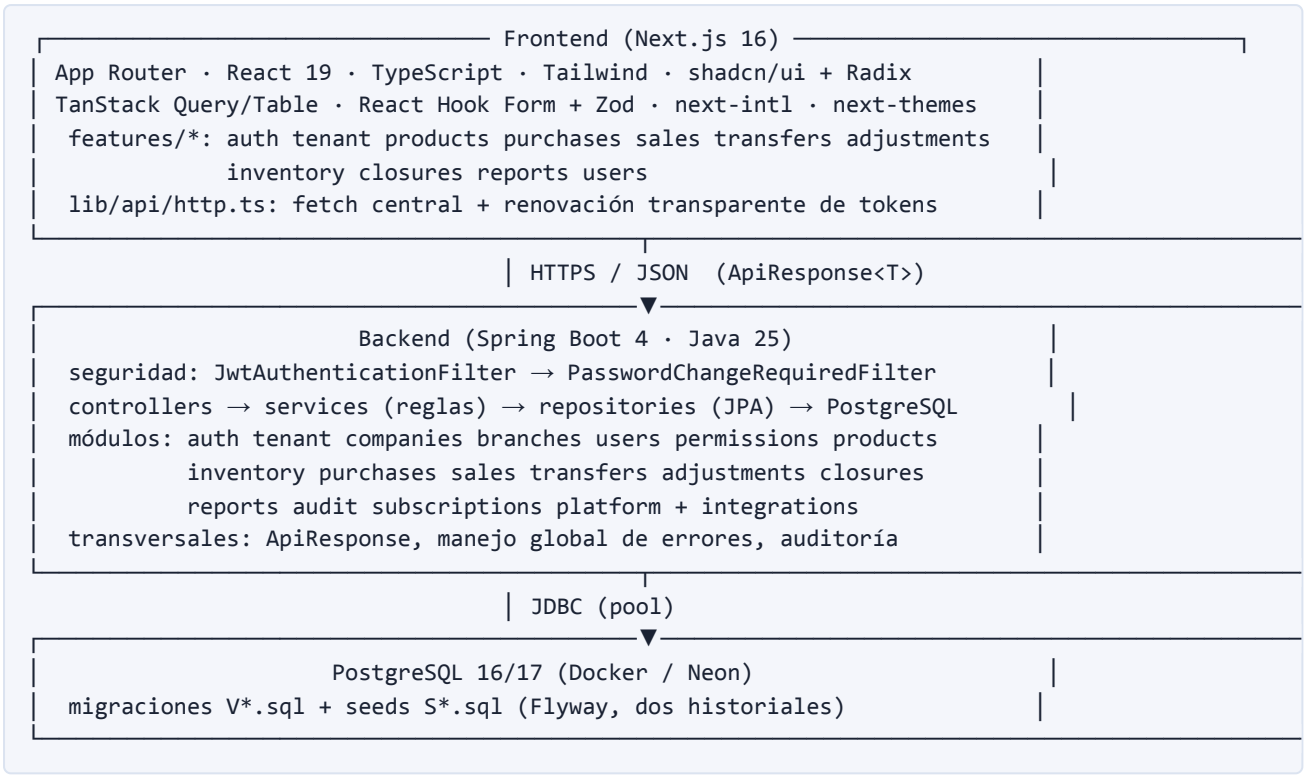
Tabla	Columnas clave	Restricciones
`purchase_details`	`id`, `purchase_id`, `presentation_id`, `quantity_presentation`, `quantity_base`, `unit_cost_base`	CHECK cantidades > 0
`sales`	`id`, `branch_id`, `customer_id`, `sale_number`, `status`, `payment_method`, `total_amount`	UNIQUE `(company_id, sale_number)`
`sale_details`	`id`, `sale_id`, `presentation_id`, `quantity_base`, `unit_price`, `discount_amount`, `fifo_cost_total`	CHECK cantidades > 0
`transfers`	`id`, `source_branch_id`, `destination_branch_id`, `status`, `requested_by`, `approved_by`, `received_by`	CHECK `source <> destination`
`transfer_details`	`id`, `transfer_id`, `quantity_requested_base`, `quantity_approved_base`, `quantity_received_base`	CHECK recibido ≤ aprobado
`inventory_adjustments`	`id`, `branch_id`, `adjustment_type`, `direction`, `status`, `reason`	CHECK dirección IN/OUT
`cash_closures`	`id`, `branch_id`, `user_id`, `closure_date`, `expected_total`, `counted_total`, `difference`, `status`	UNIQUE `(branch_id, user_id, closure_date)`
`audit_logs`	`id`, `company_id`, `branch_id`, `user_id`, `action`, `entity_type`, `entity_id`, `old_value`, `new_value`, `created_at`	Índice por entidad y fecha

Decisiones de integridad

- El saldo disponible **no se escribe a mano**: es una columna generada (`on_hand - reserved`).
- Los CHECK de cantidades impiden valores negativos, de modo que ninguna ruta de código puede dejar stock negativo.
- El cierre de caja tiene restricción única por usuario/sucursal/fecha, evitando duplicados.
- Los índices únicos por compañía (username, sku, número de compra/venta) implementan las reglas de unicidad.

7.3 Arquitectura

Patrón: monolito modular en capas. Un solo despliegue, módulos separados por dominio, dependencias en una sola dirección (controller → service → repository → entity) y reglas del negocio **solo** en la capa de servicio.



Seguridad de la arquitectura

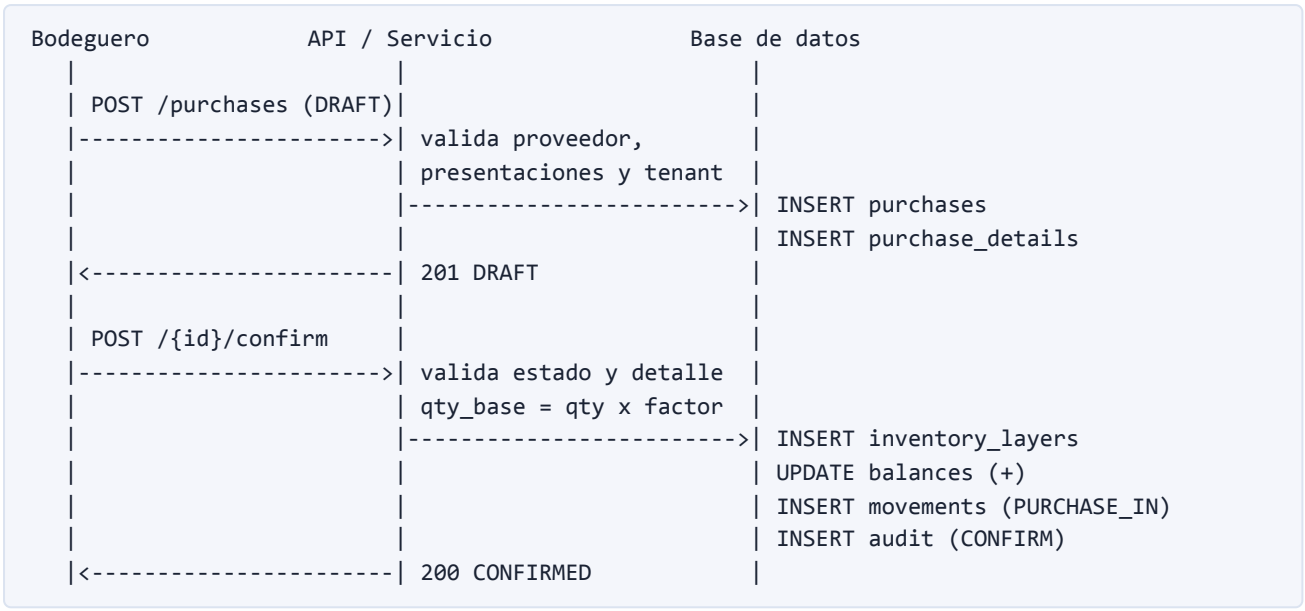
- Toda petición privada pasa por el filtro JWT y por `@PreAuthorize` con el permiso `módulo.acción`.
- El tenant se toma del token, nunca del cuerpo de la petición, y cada consulta filtra por `company_id`.
- Los usuarios sin rol COMPANY_ADMIN quedan limitados a su sucursal en la capa de servicio.
- El frontend oculta acciones sin permiso, pero **la API siempre revalida**.

7.4 Flujos de los tres procesos integrados

Cada proceso conecta al menos dos módulos y aplica reglas del negocio; su responsable se indica en el título.

P1 — Compra con entrada a inventario FIFO (Christian Méndez)

Módulos conectados: Compras + Inventario (y Catálogo como origen de presentaciones). **Actores:** bodeguero (`inventory` , rol WAREHOUSE_USER) crea y confirma; la anulación exige además `purchases.void` . **Precondiciones:** proveedor activo, presentaciones activas con unidad base, usuario con `purchases.create` y `purchases.confirm` .



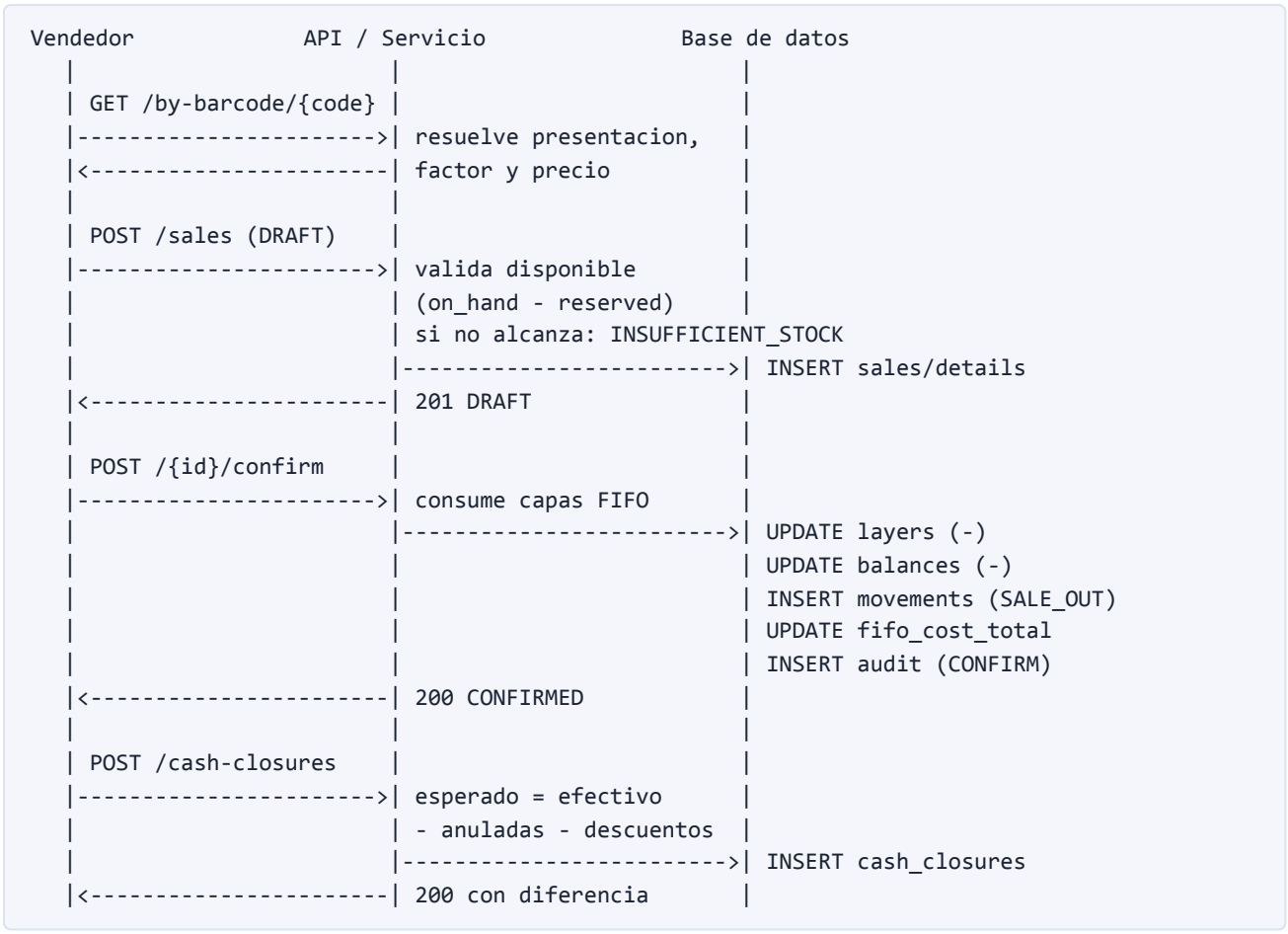
Paso a paso

1. El bodeguero abre **Compras** → **Nueva compra** y elige proveedor y fecha.
2. Agrega líneas por **presentación** (Lata, Six Pack, Caja 24), con cantidad y costo de la presentación.
3. El backend convierte cada línea a unidad base (`cantidad × factor`) y calcula subtotales.
4. Guarda como **borrador** (`DRAFT`); puede editar o eliminar el borrador.
5. Al **confirmar**, el servicio valida: estado `DRAFT`, al menos una línea, cantidades > 0, tenant y sucursal del usuario.
6. Por cada línea crea una **capa FIFO** con cantidad recibida, costo unitario en unidad base, lote y vencimiento opcionales.
7. Aumenta `quantity_on_hand` del balance de la sucursal/almacén y recalcula `quantity_available`.
8. Registra un movimiento `PURCHASE_IN` en el kardex por línea, con costo y saldo posterior.
9. Marca la compra `CONFIRMED`, guarda `confirmed_by` y `confirmed_at` y escribe la auditoría.
10. Una compra confirmada **ya no se edita**; si hubo error se anula con motivo (R19) y el sistema revierte exactamente las capas creadas por esa compra (R20).

Datos afectados: `purchases`, `purchase_details`, `inventory_layers`, `inventory_balances`, `stock_movements`, `audit_logs`. **Errores manejados:** sin líneas, producto/presentación inactiva, proveedor de otra compañía, doble confirmación, anulación sin motivo, anulación de compra ya anulada. **Evidencia:** detalle de compra con estado, kardex del producto con `PURCHASE_IN`, capas FIFO visibles con costo y saldo.

P2 — Venta en POS, consumo FIFO y cierre de caja (Wilfero Delgado)

Módulos conectados: Ventas + Inventario + Cierres de caja. **Actores:** vendedor (`seller`) opera el POS y crea el cierre; administrador aprueba el cierre. **Precondiciones:** caja abierta del día, stock disponible, usuario con `sales.create`; descuentos requieren `sales.apply_discount`.



Paso a paso

1. El vendedor abre **Ventas** → **Nueva venta**; el POS toma la sucursal de su sesión.
2. Identifica al cliente por NIT o usa **Consumidor Final**.
3. Escanea el código de barras; el backend resuelve producto, presentación, factor y precio.
4. Si el código no existe en el catálogo local, el POS consulta el **catálogo externo** (Open Food Facts) y ofrece registrar el producto con su presentación unitaria en un paso.
5. Cada línea se agrega al carrito en cantidad de presentación; el backend valida **stock disponible** antes de aceptar.
6. Aplica descuentos si el rol tiene `sales.apply_discount` (por línea o por venta).
7. Elige método de pago y **finaliza** (`DRAFT → CONFIRMED`).
8. El servicio consume capas **FIFO** de la más antigua a la más nueva, calcula el costo real (`fifo_cost_total`) y descuenta el balance.
9. Registra `SALE_OUT` en el kardex por línea y la auditoría con totales.
10. La venta queda disponible para el **cierre de caja**: vista previa con ventas confirmadas, anuladas, descuentos y totales por método de pago.
11. El vendedor registra el **monto contado**; el sistema calcula la **diferencia** y deja el cierre en estado abierto/cerrado.
12. El administrador puede **aprobar** el cierre o **reabrirlo** para recalcular si hubo correcciones.

Anulación: exige permiso `sales.cancel`, motivo obligatorio y disposición de los productos (devolver a inventario, enviar a daño, enviar a pérdida o no devolver). Devuelve o ajusta exactamente las cantidades vendidas, con movimientos `SALE_CANCEL_IN` o de ajuste. **Datos afectados:** `sales`, `sale_details`, `inventory_layers`, `inventory_balances`, `stock_movements`, `cash_closures`, `audit_logs`. **Errores**

manejados: stock insuficiente, cantidad cero o negativa, producto inactivo, descuento sin permiso, doble confirmación, anulación sin motivo, cierre duplicado del día.

P3 — Traslado entre sucursales (Fernando Palencia)

Módulos conectados: Traslados + Inventario en origen y destino. **Actores:** `transfers_destination` (WAREHOUSE_USER, sucursal 2) solicita y recibe; `admin` (COMPANY_ADMIN) aprueba; `transfers_origin` (WAREHOUSE_USER, sucursal 1) despacha. **Precondiciones:** dos sucursales activas de la **misma** compañía, stock disponible en origen, permisos `transfers.request/approve/dispatch/receive`.



Paso a paso

1. La sucursal destino crea la solicitud indicando sucursal origen, productos y cantidades (en presentación o unidad base).
2. El backend valida que ambas sucursales pertenezcan a la misma compañía (R24) y que haya detalle.
3. Un usuario con `transfers.approve` (MANAGER o COMPANY_ADMIN; en la demo, `admin`) revisa y **aprueba**: se reserva stock en origen (`quantity_reserved` sube, `quantity_available` baja) sin descontar existencia física (R22). Si `transfers_origin` intenta aprobar, la API responde `PERMISSION_DENIED`.
4. Si no hay stock suficiente para reservar, la aprobación se rechaza.
5. La sucursal origen **despacha**: el traslado pasa a `IN_TRANSIT` y se registra el movimiento de reserva/tránsito.
6. La sucursal destino **recibe**. Puede recibir **menos** de lo aprobado: el sistema guarda `quantity_received_base` y deja la diferencia como evidencia (R23), con estado `PARTIALLY_RECEIVED`.

7. Al recibir se descuenta la existencia y la reserva en origen, se suma la existencia en destino y se generan `TRANSFER_OUT` y `TRANSFER_IN` en el kardex de cada sucursal.
8. Cualquier cancelación exige motivo y libera la reserva.
9. Todo el ciclo queda auditado con usuario, fecha y cantidades.

Datos afectados: `transfers`, `transfer_details`, `inventory_balances` (origen y destino), `stock_movements`, `audit_logs`. **Errores manejados:** traslado entre compañías, sin detalle, recepción mayor a lo aprobado, aprobar un traslado ya aprobado, recibir un traslado no despachado, cancelar sin motivo.

8. Matriz de responsabilidades

Integrante	Módulos (2)	Proceso (1)	Reportes (3)	API externa (1)	Componentes compartidos
Christian Méndez	Catálogo · Inventario	P1 Compra → FIFO	Existencias por sucursal · Kardex por producto · Productos con stock bajo	**Photon** (geocodificación real)	Autenticación y permisos · Auditoría
Wilfero Delgado	Compras · Ventas POS	P2 Venta → Cierre	Ventas por período · Compras por proveedor · Cierre de caja	**Open Food Facts** (barcode externo)	Dashboard · Docker y publicación
Fernando Palencia	Traslados · Ajustes	P3 Traslado entre sucursales	Traslados por estado · Ajustes de inventario · Movimientos de inventario	**open.er-api.com** (tipo de cambio)	Datos de prueba · Revisión de seguridad

Transversales distribuidos: login (los tres), dashboard (Wilfero), publicación (Wilfero + Christian), datos de prueba (Fernando), documentación y evidencias (los tres).

9. Stack tecnológico y equivalencia

Capa	Tecnología	Justificación
Base de datos	PostgreSQL 16/17	Coincide con el Stack 2 del enunciado.
Backend	Java 25 LTS + Spring Boot 4	Equivalente funcional a NestJS: API REST, inyección de dependencias, validación, seguridad, ORM y migraciones. Ver `JUSTIFICACION_STACK_SPRING_BOOT.md`.
Frontend	Next.js 16 (React 19) + TypeScript	Coincide con el Stack 2.
Estilos	Tailwind CSS + shadcn/ui + Radix	HTML/CSS exigido, con accesibilidad.
Estado y formularios	TanStack Query/Table, React Hook Form, Zod	Consultas, tablas y validación en frontend.
i18n / tema	next-intl, next-themes	es-GT / en-US, claro / oscuro.

Capa	Tecnología	Justificación
Contenedores	Docker + Docker Compose	BD local y stack completo.
Pruebas de API	Swagger/OpenAPI + Postman	Colección por módulo, auth e integraciones.
Control de versiones	Git + GitHub	Historial de los tres integrantes.
Agente de IA	`AGENTS.md`, `SKILL.md`, `memory.md`	Instrucciones, habilidades y memoria del proyecto.

Solicitud de equivalencia: el proyecto usa Spring Boot en lugar de NestJS. El alcance funcional, la seguridad, la arquitectura y la documentación son equivalentes; la solicitud formal está en `JUSTIFICACION_STACK_SPRING_BOOT.md`.

10. Repositorio, Docker y publicación

Repositorio: `https://github.com/wilfero/vivent-academico` (privado; el docente recibe acceso). Estructura: `backend/`, `frontend/`, `database/`, `docker/`, `docs/`, `postman/`, `README.md`, `.env.example`, `.gitignore`, `AGENTS.md`, `SKILL.md`, `memory.md`.

Arranque local con Docker (una sola orden):

```
cd docker
docker compose -f compose.local.yml up -d --build
```

Servicios: PostgreSQL en `localhost:5433`, backend en `localhost:8080`, frontend en `localhost:3000`. En el primer arranque el backend aplica migraciones y seeds automáticamente.

Operaciones demo (opcional, idempotente):

```
python database/bootstrap_demo.py
```

Crea compras, ventas con descuento, un traslado completo, un ajuste aprobado y el cierre del día a través de la API real.

Publicación en la nube: guía en `PUBLICACION_CLOUD.md` (Neon + Render + Vercel), con evidencia de conexión y cuentas de prueba.

11. API, colección Postman y CRUD

La API REST de Vivent expone **132 endpoints** bajo el prefijo `/api/v1`. Todas las respuestas usan el mismo sobre (`success`, `data`, `message`, `errorCode`), la autenticación es **JWT Bearer con access token y refresh token** y cada endpoint privado vuelve a validar en el servidor: token vigente, `company_id` del token, usuario activo, suscripción habilitada, sucursal cuando aplica y el permiso `module.action` que corresponda. El frontend nunca es la única barrera.

La documentación interactiva está en `http://localhost:8080/swagger-ui/index.html`. La colección Postman (`postman/Vivent.postman_collection.json`) cubre el ciclo completo con cuerpos listos para ejecutar y aserciones que verifican cada respuesta.

11.1 Instalación de la colección

1. **Importar los dos archivos** de la carpeta `postman/`: la colección y el environment `Vivent-Local.postman_environment.json`.
2. **Seleccionar el environment** `Vivent Local` en la esquina superior derecha de Postman.
3. **Ejecutar** `01 · Autenticación → Login (guarda tokens)`. El login escribe `accessToken`, `refreshToken`, `branchId`, `companyId` y `userId` en las variables de la colección; desde ahí el resto de requests ya viajan autenticados.

Si prefiere ejecutarla sin abrir Postman:

```
npx newman run postman/Vivent.postman_collection.json \
  -e postman/Vivent-Local.postman_environment.json
```

> El environment **no** declara `accessToken` a propósito: si lo declarara vacío, tendría prioridad sobre la variable de colección y todos los requests responderían 401. Es el error de configuración más común al importar colecciones.

11.2 Variables de la colección

Variables de entrada (las únicas que se editan a mano):

Variable	Valor por defecto	Para qué sirve
`baseUrl`	`http://localhost:8080`	URL de la API (cámbiela al publicar en otro host).
`tenantSlug`	`demo`	Subdominio de la empresa; define el tenant.
`username`	`admin`	Usuario demo.
`password`	`ChangeMe123!`	Contraseña demo (solo ambiente local).

Variables que se llenan solas durante la corrida:

Variable	La escribe	La consumen
`accessToken`, `refreshToken`	Login y refresh	Todos los requests privados.
`companyId`, `branchId`, `userId`, `roleCode`	Login	Contexto del token y permisos.
`branch2Id`	`GET Sucursales`	Traslados entre sucursales.
`categoryId`, `productId`, `presentationId`, `comboId`	Catálogo	Compras, ventas, traslados, ajustes.
`supplierId`, `customerId`	Terceros	Compra y venta.

Variable	La escribe	La consumen
`purchaseId`, `saleId`, `transferId`, `transferDetailId`, `adjustmentId`, `closureId`	Operaciones	Confirmar, anular, recibir, aprobar.
`roleId`, `newUserId`	Roles y usuarios	Alta de usuario y perfil.
`tempCategoryId`, `tempProductId`, `tempPresentationId`, `tempSupplierId`	Registros temporales	Demostrar la baja lógica sin romper la cadena.
`today`, `yesterday`, `nowIso`, `closureDate`	Pre-request de la colección	Filtros de reportes y fecha del cierre.
`runId`	Login	Sufijo único por corrida: nombres, SKU, códigos de barras y números de documento.

Gracias a `runId`, la colección se puede ejecutar **varias veces seguidas** sin chocar con SKU, códigos de barras o números de documento ya existentes.

11.3 Estructura de la colección

Carpeta	Requests	Contenido
00 · Público	1	Configuración del tenant por subdominio (sin token).
01 · Autenticación	5	Login, `/auth/me`, refresh con rotación, logout y login fallido.
02 · Configuración del tenant	6	Empresa, sucursales y suscripción.
03 · Catálogo	31	Categorías, unidades, productos, presentaciones, combos y búsqueda por código de barras.
04 · Terceros	11	Proveedores y clientes.
05 · Inventario	14	Balances, capas FIFO, kardex, movimientos, entradas/salidas y ajustes.
06 · Compras	6	Borrador, actualización, confirmación y anulación.
07 · Ventas	8	CRUD del borrador, confirmación (FIFO) y anulación.
08 · Traslados	9	Solicitar, aprobar, despachar, recibir y cancelar.
09 · Cierres de caja	7	Preview, alta, aprobación, reapertura y cierre con monto contado.
10 · Reportes	6	Los seis reportes con filtros de fecha.
11 · Integraciones externas	6	Photon, Open Food Facts y tipo de cambio (éxito y error).
12 · Usuarios y permisos	17	Roles, usuarios, overrides y perfil propio.
13 · Auditoría	1	Bitácora de acciones sensibles.
Total	**128**	Cobertura de todos los módulos del sistema.

11.4 Matriz CRUD por recurso

Todas las rutas llevan el prefijo `/api/v1`. Ninguna entidad se borra físicamente: la columna «Baja» indica la **baja lógica** (estado o `deleted_at`) o la anulación con motivo.

Recurso	Crear	Leer	Actualizar	Baja / anulación
Categorías	`POST /categories`	`GET /categories`, `GET /categories/{id}`	`PUT /categories/{id}`	`DELETE /categories/{id}`
Unidades	`POST /product-units`	`GET /product-units`	`PUT /product-units/{code}`	`DELETE /product-units/{code}`
Productos	`POST /products`	`GET /products`, `GET /products/{id}`	`PUT /products/{id}`	`DELETE /products/{id}`
Presentaciones	`POST /products/{id}/presentations`	`GET /products/{id}/presentations`	`PUT /products/{id}/presentations/{pid}`	`DELETE /products/{id}/presentations/{pid}`
Combos	`POST /combos`	`GET /combos`, `GET /combos/{id}`, `GET /combos/by-barcode/{barcode}`	`PUT /combos/{id}`	`DELETE /combos/{id}`
Proveedores	`POST /suppliers`	`GET /suppliers`, `GET /suppliers/{id}`	`PUT /suppliers/{id}`	`DELETE /suppliers/{id}`
Clientes	`POST /customers`	`GET /customers`, `GET /customers/{id}`	`PUT /customers/{id}`	`DELETE /customers/{id}`
Compras	`POST /purchases`	`GET /purchases`, `GET /purchases/{id}`	`PUT /purchases/{id}` (solo DRAFT)	`POST /purchases/{id}/void`
Ventas	`POST /sales`	`GET /sales`, `GET /sales/{id}`	`PUT /sales/{id}` (solo DRAFT)	`DELETE /sales/{id}`, `POST /sales/{id}/cancel`
Traslados	`POST /transfers`	`GET /transfers`, `GET /transfers/{id}`	`POST /transfers/{id}/approve`, `/dispatch`, `/receive`	`POST /transfers/{id}/cancel`
Ajustes	`POST /inventory/adjustments`	`GET /inventory/adjustments`, `GET /inventory/adjustments/{id}`	`PUT /inventory/adjustments/{id}`	`POST /{id}/approve`, `/reject`, `/cancel`

Recurso	Crear	Leer	Actualizar	Baja / anulación
Cierres de caja	`POST /cash-closures`	`GET /cash-closures`, `GET /cash-closures/{id}`, `GET /cash-closures/preview`	`POST /cash-closures/{id}/reopen`, `/close`	`POST /cash-closures/{id}/approve`
Roles	`POST /roles`	`GET /roles`, `GET /roles/{id}`, `GET /permissions`	`PUT /roles/{id}`	— (los roles de sistema no se borran)
Usuarios	`POST /users`	`GET /users`, `GET /users/{id}`, `GET /users/me`	`PUT /users/{id}`, `PUT /users/me`	`POST /users/{id}/deactivate`, `DELETE /users/{id}`
Empresa y sucursales	—	`GET /companies/current`, `GET /branches`	`PUT /companies/current`, `PUT /branches/{id}`	—

Cada operación exige el permiso `module.action` del módulo correspondiente (`products.*`, `product_categories.*`, `product_presentations.*`, `units.*`, `combos.*`, `suppliers.*`, `customers.*`, `purchases.*`, `sales.*`, `transfers.*`, `inventory.adjust` y `inventory.adjust_approve`, `cash_closures.*`, `users.*`, `settings.*`). Un usuario sin el permiso recibe `403 PERMISSION_DENIED` aunque el botón exista en pantalla.

11.5 Corrida encadenada de una prueba completa

Una corrida de la colección reproduce el ciclo operativo real, en este orden:

1. **Login** como `admin` → se guardan tokens e ids del contexto.
2. **Catálogo**: categoría → producto → presentación (y combo). La presentación queda como `presentationId`.
3. **Terceros**: proveedor y cliente nuevos.
4. **Inventario**: entrada manual de stock y consulta de balances, capas FIFO y kardex.
5. **Compra**: crear borrador → actualizar → confirmar. Al confirmar se crea la capa FIFO, sube la existencia y se registra kardex + auditoría.
6. **Venta**: CRUD de un borrador (crear, leer, actualizar, eliminar) y luego una venta nueva que se confirma consumiendo FIFO y se anula devolviendo la mercadería a inventario.
7. **Traslado**: solicitar en la sucursal 2 → aprobar (reserva stock) → despachar → recibir.
8. **Ajuste**: crear por daño → aprobar (genera kardex de salida).
9. **Cierre**: preview del día → crear cierre → aprobar → reabrir → cerrar con monto contado.
10. **Reportes y auditoría**: los seis reportes con el rango de fechas y la bitácora de todo lo anterior.

11.6 Errores controlados y resultado de la corrida

La colección incluye a propósito casos de error para demostrar que el backend valida reglas, no solo el formulario. Sus aserciones esperan un 4xx y `success = false`, por lo que **no cuentan como fallo**:

Request	Respuesta esperada	Regla que demuestra
Login con clave incorrecta	`401 INVALID_CREDENTIALS`	Autenticación por tenant + usuario.
Refresh reutilizado	`401`	Rotación y revocación de refresh tokens.
Rechazar ajuste ya aprobado	`400 INVALID_*_STATUS`	Máquina de estados del ajuste.
Cancelar traslado ya recibido	`400 INVALID_TRANSFER_STATUS`	Máquina de estados del traslado.
Cambiar contraseña por la misma	`400 VALIDATION_ERROR`	Política de contraseñas.

Resultado verificado de la corrida completa (ejecutada dos veces seguidas sobre la misma base para comprobar que es repetible):

Métrica	Valor
Requests ejecutados	128
Aserciones ejecutadas	242
Aserciones fallidas	0
Duración aproximada	25 s
Respuesta promedio	65 ms

Las evidencias de estas corridas están en `docs/evidencias/evidencias_pruebas.md`.

12. Anexos

Cuentas demo (contraseña `ChangeMe123!`, solo ambiente de prueba):

Usuario	Rol	Uso demostrativo
`admin`	COMPANY_ADMIN	Administración completa; aprueba traslados, ajustes y cierres; reportes y auditoría
`seller`	SELLER	POS y cierre de caja
`inventory`	WAREHOUSE_USER	Compras (crear/confirmar) y ajustes
`transfers_origin`	WAREHOUSE_USER	Despacho de traslados de la sucursal 1
`transfers_destination`	WAREHOUSE_USER (sucursal 2)	Solicitar y recibir traslados
`auditor`	AUDITOR	Solo lectura de reportes y bitácora

Documentos complementarios

- `MATRIZ_CUMPLIMIENTO_DESARROLLO_WEB.md` — requisito por requisito con evidencia.
- `MANUAL_TECNICO.md` — instalación, configuración, estructura y APIs externas.
- `MANUAL_USUARIO.md` — guía ilustrada de los procesos principales.
- `APIS_EXTERNAS.md` — las tres integraciones con ejemplos de éxito y error.
- `PUBLICACION_CLOUD.md` — publicación y checklist de entrega.

- `evidencias/evidencias_pruebas.md` — casos y resultados de pruebas.
- `postman/Vivent.postman_collection.json` — colección con 128 requests (CRUD completo).
- `postman/Vivent-Local.postman_environment.json` — environment con URL y credenciales demo.
- `postman/README.md` — instalación, orden de ejecución y tabla de variables.

---

13. Rúbrica. La hoja original de la rúbrica del curso se adjunta al final de este documento, tal como la entrega el catedrático.